

Modelos de Mezcla Gaussiana en Imágenes RGB

Guía de estudio: agrupamiento de colores, EM, cuantización y clasificación probabilística

FCFM–UNACH
Licenciatura en Matemáticas Aplicadas
Curso de Aprendizaje Máquina Estadístico

Índice

1. Propósito de la guía	3
2. De una imagen a un conjunto de datos	3
3. Motivación: agrupamiento de colores	4
4. Modelo de Mezcla Gaussiana para RGB	4
4.1. Interpretación de los parámetros	5
5. Variable latente y pertenencia a componentes	5
6. Algoritmo Expectation–Maximization	5
6.1. Paso E: estimación de responsabilidades	6
6.2. Paso M: actualización de parámetros	6
7. Simplificación pedagógica: covarianzas diagonales	7
8. Ejemplo mínimo: seis píxeles	7
9. Inicialización del ejemplo	8
9.1. ¿Por qué comenzar con valores intuitivos?	8
10. Primera iteración: paso E	8
10.1. Cómo leer la matriz de responsabilidades	9
11. Primera iteración: paso M	9
12. Log-verosimilitud y progreso del algoritmo	10
13. De seis píxeles a una imagen sintética	11
13.1. ¿Por qué añadir ruido?	12
14. Vectorización de la imagen	12
15. Ajuste automático con mclust	13
15.1. Selección mediante BIC	13
16. Responsabilidades y medias estimadas	13
17. Cuantización MAP	14
17.1. Interpretación como compresión de colores	14

18.Reconstrucción mediante media posterior	15
18.1. MAP frente a promedio posterior	15
19.Visualización comparativa	15
20.Geometría RGB y reducción de dimensión con PCA	16
20.1. ¿Qué representa esta gráfica?	17
21.Uso de una imagen real	17
21.1. Qué debe analizarse en una imagen real	18
22.Una limitación fundamental: se ignora la posición	19
23.Comparación conceptual con k-means	19
24.Evaluación de la reconstrucción	20
25.Preguntas de autoevaluación	20
26.Ejercicios propuestos	21
27.Resumen conceptual	22
28.Bibliografía recomendada	22

1. Propósito de la guía

Una imagen digital en color puede interpretarse no sólo como una colección de objetos visuales, sino también como un conjunto de datos. Cada píxel contiene información numérica sobre intensidad de color y, en una imagen RGB, puede representarse mediante tres coordenadas: rojo, verde y azul.

Esta perspectiva permite aplicar herramientas de aprendizaje máquina estadístico directamente sobre los píxeles. En particular, los **Modelos de Mezcla Gaussiana** (*Gaussian Mixture Models*, GMM) permiten identificar regiones del espacio de color que aparecen con frecuencia en una imagen, estimar probabilidades de pertenencia de cada píxel a dichas regiones y construir versiones simplificadas de la imagen utilizando una paleta de colores aprendida a partir de los datos.

El objetivo de esta guía es desarrollar esas ideas de manera progresiva. Primero se trabajará con seis píxeles para realizar una iteración del algoritmo EM de forma transparente. Después, la misma lógica se trasladará a una imagen sintética completa y finalmente se utilizará el paquete `mclust` para automatizar el ajuste.

Objetivos de aprendizaje

Al finalizar este material, el estudiante deberá ser capaz de:

1. Representar una imagen RGB como una muestra de vectores en $\mathbb{R}^3$.
2. Formular un GMM para modelar colores de píxeles.
3. Interpretar las variables latentes y las responsabilidades del algoritmo EM.
4. Realizar una iteración E–M con covarianzas diagonales.
5. Explicar la diferencia entre cuantización MAP y promedio posterior.
6. Ajustar un GMM a los píxeles de una imagen mediante `mclust`.
7. Interpretar el número de componentes y las medias Gaussianas como una paleta de colores aprendida.
8. Utilizar PCA para visualizar la geometría de los colores en dos dimensiones.

2. De una imagen a un conjunto de datos

En el modelo RGB, cada color está determinado por tres intensidades. Un píxel puede escribirse como

$$x_i = \begin{pmatrix} R_i \\ G_i \\ B_i \end{pmatrix}.$$

Cuando las intensidades se almacenan con 8 bits por canal, cada coordenada pertenece al conjunto

$$\{0, 1, \dots, 255\}.$$

Para trabajar estadísticamente resulta conveniente normalizar:

$$(R_i, G_i, B_i) \mapsto \left(\frac{R_i}{255}, \frac{G_i}{255}, \frac{B_i}{255} \right),$$

de modo que

$$x_i \in [0, 1]^3.$$

Si una imagen posee H filas y W columnas, entonces contiene

$$N = HW$$

píxeles. Desde el punto de vista estadístico, puede reorganizarse como una matriz

$$X = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} \in \mathbb{R}^{N \times 3}.$$

Cada fila es una observación y las tres columnas corresponden a los canales R , G y B .

Interpretación

Esta representación cambia la pregunta. En lugar de preguntar “¿qué objetos aparecen en la imagen?”, podemos preguntar “¿qué grupos de colores aparecen entre sus píxeles y con qué frecuencia?”. Un GMM responde a esta segunda pregunta.

3. Motivación: agrupamiento de colores

Muchas imágenes contienen conjuntos de píxeles concentrados alrededor de ciertos colores dominantes. Por ejemplo, una fotografía podría contener regiones asociadas con cielo, vegetación, suelo y construcciones.

Si los píxeles de una misma región cromática tienen valores RGB similares, entonces forman una nube de puntos en el cubo

$$[0, 1]^3.$$

Una estrategia natural es aproximar la distribución total de los colores mediante una combinación de varias distribuciones Gaussianas. Cada componente tendrá:

- un peso π_k , que representa su frecuencia relativa;
- una media $\mu_k \in \mathbb{R}^3$, interpretable como un color representativo;
- una covarianza $\Sigma_k \in \mathbb{R}^{3 \times 3}$, que describe la variabilidad de los colores alrededor de esa media.

Conexión con aprendizaje máquina

El modelo convierte un problema visual en un problema estadístico no supervisado. No proporcionamos etiquetas como “rojo”, “vegetación” o “cielo”. El algoritmo únicamente observa los vectores RGB y busca una representación probabilística de su distribución.

4. Modelo de Mezcla Gaussiana para RGB

Sea

$$X \in \mathbb{R}^3$$

el vector aleatorio que representa un píxel normalizado. Un GMM con K componentes supone que su densidad tiene la forma

$$p(x \mid \Theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \quad (1)$$

donde

$$\pi_k > 0, \quad \sum_{k=1}^K \pi_k = 1.$$

El conjunto de parámetros es

$$\Theta = \{\pi_k, \mu_k, \Sigma_k : k = 1, \dots, K\}.$$

4.1. Interpretación de los parámetros

La media del componente k ,

$$\mu_k = \begin{pmatrix} \mu_{k,R} \\ \mu_{k,G} \\ \mu_{k,B} \end{pmatrix},$$

puede interpretarse como el color central de ese grupo. Si, por ejemplo,

$$\mu_k = (0.90, 0.10, 0.10),$$

entonces el componente está centrado alrededor de un rojo intenso.

La matriz Σ_k determina la forma de la nube de colores alrededor de μ_k . Una mayor varianza implica una gama cromática más amplia.

5. Variable latente y pertenencia a componentes

El componente que generó cada píxel no se observa directamente. Por ello introducimos una variable latente

$$Z_i \in \{1, \dots, K\}.$$

La interpretación es

$$Z_i = k \iff x_i \text{ fue generado por el componente } k.$$

Los pesos satisfacen

$$P(Z_i = k) = \pi_k.$$

Condicionalmente a $Z_i = k$,

$$X_i \mid (Z_i = k) \sim \mathcal{N}(\mu_k, \Sigma_k).$$

El problema consiste entonces en estimar simultáneamente los parámetros de los componentes y la pertenencia probabilística de cada píxel.

6. Algoritmo Expectation–Maximization

El algoritmo EM resuelve este problema alternando dos etapas.

6.1. Paso E: estimación de responsabilidades

Definimos

$$\gamma_{ik} = P(Z_i = k \mid X_i = x_i, \Theta).$$

Usando el teorema de Bayes,

$$\gamma_{ik} = \frac{\pi_k \phi_3(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \phi_3(x_i; \mu_j, \Sigma_j)}. \quad (2)$$

Para cada píxel i ,

$$\sum_{k=1}^K \gamma_{ik} = 1.$$

Por tanto, el vector

$$(\gamma_{i1}, \dots, \gamma_{iK})$$

forma una distribución de probabilidad sobre los componentes.

Interpretación

Las responsabilidades constituyen una clasificación *blanda*. Un píxel situado entre dos regiones cromáticas puede recibir, por ejemplo,

$$(0.52, 0.46, 0.02),$$

en lugar de ser obligado inmediatamente a pertenecer a un único grupo.

6.2. Paso M: actualización de parámetros

Primero se calcula el tamaño efectivo de cada componente:

$$N_k = \sum_{i=1}^N \gamma_{ik}.$$

Después,

$$\pi_k^{\text{new}} = \frac{N_k}{N}, \quad (3)$$

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} x_i, \quad (4)$$

y

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (x_i - \mu_k^{\text{new}})(x_i - \mu_k^{\text{new}})^\top. \quad (5)$$

Así, cada observación contribuye a cada componente en proporción a su responsabilidad.

7. Simplificación pedagógica: covarianzas diagonales

Para realizar la primera iteración manualmente se supondrá

$$\Sigma_k = \begin{pmatrix} \sigma_{k,R}^2 & 0 & 0 \\ 0 & \sigma_{k,G}^2 & 0 \\ 0 & 0 & \sigma_{k,B}^2 \end{pmatrix}.$$

Entonces la densidad Gaussiana tridimensional se factoriza:

$$\phi_3(x; \mu_k, \Sigma_k) = \phi(R; \mu_{k,R}, \sigma_{k,R}) \phi(G; \mu_{k,G}, \sigma_{k,G}) \phi(B; \mu_{k,B}, \sigma_{k,B}). \quad (6)$$

Esta formulación permite calcular la densidad utilizando tres normales unidimensionales.

Punto de atención

La diagonalidad es una simplificación para comprender EM. En imágenes reales puede existir correlación entre canales. Un modelo con covarianzas completas puede representar nubes inclinadas en el espacio RGB y, por tanto, estructuras cromáticas más flexibles.

8. Ejemplo mínimo: seis píxeles

Antes de ajustar miles de píxeles, trabajaremos con seis observaciones representativas de tres colores.

Píxel	R	G	B
1	230	30	35
2	240	35	40
3	20	220	25
4	25	210	35
5	30	35	230
6	28	40	240

En R:

```
1 RGB_255 <- matrix(c(
2   230, 30, 35,
3   240, 35, 40,
4   20, 220, 25,
5   25, 210, 35,
6   30, 35, 230,
7   28, 40, 240
8 ), ncol = 3, byrow = TRUE)
9
10 colnames(RGB_255) <- c("R", "G", "B")
11
12 X <- RGB_255 / 255
13 DF <- data.frame(i = 1:nrow(X), round(X, 4))
14 DF
```

La normalización es importante porque coloca los tres canales en el intervalo común $[0, 1]$.

9. Inicialización del ejemplo

Usaremos $K = 3$ componentes y pesos iguales:

$$\pi_1 = \pi_2 = \pi_3 = \frac{1}{3}.$$

Las medias iniciales serán

$$\mu_1 = (0.9, 0.1, 0.1),$$

$$\mu_2 = (0.1, 0.9, 0.1),$$

$$\mu_3 = (0.1, 0.1, 0.9).$$

Estas medias corresponden aproximadamente a rojo, verde y azul.

Usaremos además

$$\sigma_k = (0.08, 0.08, 0.08)$$

para los tres componentes.

```
1 K <- 3
2
3 pi <- rep(1/K, K)
4
5 mu <- rbind(
6   c(0.9, 0.1, 0.1),
7   c(0.1, 0.9, 0.1),
8   c(0.1, 0.1, 0.9)
9 )
10
11 sd <- rbind(
12   c(0.08, 0.08, 0.08),
13   c(0.08, 0.08, 0.08),
14   c(0.08, 0.08, 0.08)
15 )
```

9.1. ¿Por qué comenzar con valores intuitivos?

EM necesita un punto inicial. En problemas complejos, distintas inicializaciones pueden llevar a soluciones locales diferentes. Aquí elegimos valores fácilmente interpretables para concentrarnos en el mecanismo del algoritmo.

10. Primera iteración: paso E

Para cada píxel y componente calculamos

$$a_{ik} = \pi_k \phi_3(x_i; \mu_k, \Sigma_k).$$

Después normalizamos:

$$\gamma_{ik} = \frac{a_{ik}}{\sum_{j=1}^K a_{ij}}.$$

La densidad diagonal se implementa mediante:

```

1 d3_diag <- function(x, mu, sd){
2   dnorm(x[1], mu[1], sd[1]) *
3   dnorm(x[2], mu[2], sd[2]) *
4   dnorm(x[3], mu[3], sd[3])
5 }

```

Las responsabilidades se calculan con:

```

1 dens <- sapply(
2   1:K,
3   function(k)
4     apply(X, 1, function(x)
5       d3_diag(x, mu[k,], sd[k,]))
6 )
7
8 num <- sweep(dens, 2, pi, '*')
9 den <- rowSums(num)
10
11 Gamma <- num / den
12
13 Gamma_round <- round(Gamma, 4)
14 cbind(DF, Gamma_round)

```

10.1. Cómo leer la matriz de responsabilidades

La matriz Γ tiene dimensiones

$$N \times K.$$

La fila i contiene las probabilidades de pertenencia del píxel i , mientras que la columna k contiene las responsabilidades asociadas con el componente k .

Si una fila fuese

$$(0.98, 0.01, 0.01),$$

el primer componente explicaría casi completamente ese píxel. En cambio,

$$(0.45, 0.50, 0.05)$$

indicaría un color situado en una región de solapamiento entre los componentes 1 y 2.

Actividad para el estudiante

Seleccione uno de los seis píxeles y calcule manualmente los tres productos

$$\pi_k \phi(R_i; \mu_{k,R}, \sigma_{k,R}) \phi(G_i; \mu_{k,G}, \sigma_{k,G}) \phi(B_i; \mu_{k,B}, \sigma_{k,B}).$$

Después normalice los tres valores para recuperar la fila correspondiente de Γ .

11. Primera iteración: paso M

Una vez conocidas las responsabilidades,

$$N_k = \sum_i \gamma_{ik}$$

mide el número efectivo de píxeles asignados al componente k .

En R:

```
1 Nk <- colSums(Gamma)
2
3 pi_n <- Nk / nrow(X)
4
5 mu_n <- t(Gamma) %*% X / Nk
```

La actualización de la varianza diagonal se realiza canal por canal:

```
1 var_n <- matrix(0, nrow = K, ncol = 3)
2
3 for(k in 1:K){
4   Xc <- sweep(X, 2, mu_n[k,], '-')
5   var_n[k,] <-
6     colSums(Gamma[,k] * Xc^2) / Nk[k]
7 }
8
9 sd_n <- sqrt(var_n)
```

Podemos organizar los nuevos parámetros:

```
1 Ms <- data.frame(
2   componente = paste0(1:K),
3   pi_k = pi_n,
4   mu_R = mu_n[,1],
5   mu_G = mu_n[,2],
6   mu_B = mu_n[,3],
7   sd_R = sd_n[,1],
8   sd_G = sd_n[,2],
9   sd_B = sd_n[,3]
10 )
11
12 nums <- sapply(Ms, is.numeric)
13 Ms_round <- Ms
14
15 Ms_round[,nums] <-
16   lapply(Ms_round[,nums],
17     function(z) round(z, 4))
18
19 Ms_round
```

Interpretación

Las nuevas medias son colores estimados a partir de los datos. Ya no son simplemente los valores elegidos por el usuario al inicializar. EM desplaza cada media hacia los píxeles que tienen mayor responsabilidad en ese componente.

12. Log-verosimilitud y progreso del algoritmo

Para observaciones independientes,

$$L(\Theta) = \prod_{i=1}^N p(x_i | \Theta).$$

Se utiliza la log-verosimilitud

$$\ell(\Theta) = \sum_{i=1}^N \log \left[\sum_{k=1}^K \pi_k \phi_3(x_i; \mu_k, \Sigma_k) \right]. \quad (7)$$

Una propiedad esencial de EM es que sus iteraciones no disminuyen la log-verosimilitud bajo las actualizaciones estándar:

$$\ell(\Theta^{(t+1)}) \geq \ell(\Theta^{(t)}).$$

Implementamos:

```

1 loglik <- function(X, pi, mu, sd){
2   dens <- sapply(
3     1:length(pi),
4     function(k)
5       apply(X, 1, function(x)
6         d3_diag(x, mu[k,], sd[k,]))
7   )
8
9   sum(log(
10     rowSums(
11       sweep(dens, 2, pi, '*')
12     )
13   ))
14 }
15
16 ll0 <- loglik(X, pi, mu, sd)
17 ll1 <- loglik(X, pi_n, mu_n, sd_n)
18
19 c(
20   ll_inicial = round(ll0, 4),
21   ll_despues_EM = round(ll1, 4)
22 )

```

Punto de atención

El objetivo no es que la log-verosimilitud sea positiva. El criterio relevante es comparar iteraciones. Si aumenta, el nuevo conjunto de parámetros explica mejor los datos en términos de verosimilitud.

13. De seis píxeles a una imagen sintética

El cálculo anterior contiene toda la lógica esencial. La diferencia al trabajar con una imagen completa es únicamente el número de observaciones.

Construiremos una imagen de tamaño

$$H \times W = 80 \times 120$$

con tres grandes regiones cromáticas: roja, verde y azul. Cada región tendrá pequeñas perturbaciones aleatorias para que los píxeles no sean idénticos.

```

1 set.seed(7)
2
3 H <- 80
4 W <- 120
5
6 img <- array(0, dim = c(H, W, 3))

```

```

7
8 # Bloque rojo
9 img[, 1:(W/3), 1] <-
10   0.85 + rnorm(H*W/3, 0, 0.03)
11
12 img[, 1:(W/3), 2] <-
13   0.10 + rnorm(H*W/3, 0, 0.02)
14
15 img[, 1:(W/3), 3] <-
16   0.10 + rnorm(H*W/3, 0, 0.02)
17
18 # Bloque verde
19 img[, (W/3+1):(2*W/3), 1] <-
20   0.10 + rnorm(H*W/3, 0, 0.02)
21
22 img[, (W/3+1):(2*W/3), 2] <-
23   0.85 + rnorm(H*W/3, 0, 0.03)
24
25 img[, (W/3+1):(2*W/3), 3] <-
26   0.10 + rnorm(H*W/3, 0, 0.02)
27
28 # Bloque azul
29 img[, (2*W/3+1):W, 1] <-
30   0.10 + rnorm(H*W/3, 0, 0.02)
31
32 img[, (2*W/3+1):W, 2] <-
33   0.10 + rnorm(H*W/3, 0, 0.02)
34
35 img[, (2*W/3+1):W, 3] <-
36   0.85 + rnorm(H*W/3, 0, 0.03)
37
38 # Recorte al intervalo [0,1]
39 img[img < 0] <- 0
40 img[img > 1] <- 1
41
42 grid::grid.raster(
43   img,
44   interpolate = FALSE
45 )

```

13.1. ¿Por qué añadir ruido?

Si todos los píxeles de cada bloque fueran exactamente iguales, existirían sólo tres colores y el problema sería trivial. El ruido genera nubes alrededor de los centros cromáticos y reproduce mejor la variabilidad que puede encontrarse en una imagen real por iluminación, textura, sensores o compresión.

14. Vectorización de la imagen

Para utilizar un algoritmo estadístico convertimos el arreglo tridimensional en una matriz:

```

1 Ximg <- matrix(img, ncol = 3)
2 dim(Ximg)

```

Ahora

$$X_{\text{img}} \in \mathbb{R}^{(HW) \times 3}.$$

Cada fila contiene un píxel.

Interpretación

La transformación no altera los colores; únicamente cambia la organización de los datos. Después del ajuste, los resultados deben volver a colocarse en la geometría $H \times W$ de la imagen.

15. Ajuste automático con `mclust`

Para una imagen con miles de píxeles no tiene sentido ejecutar EM manualmente. Utilizaremos el paquete `mclust`.

```
1 # install.packages("mclust")
2 library(mclust)
3
4 fit <- Mclust(
5   Ximg,
6   G = 3:6
7 )
8
9 summary(fit)
10
11 K_hat <- fit$G
12
13 cat(
14   "Numero de componentes elegido (BIC):",
15   K_hat,
16   "\n"
17 )
```

La instrucción

```
1 G = 3:6
```

indica que se consideren modelos con entre tres y seis componentes.

15.1. Selección mediante BIC

El criterio BIC equilibra ajuste y complejidad. De manera conceptual,

$$\text{BIC} = 2 \ell(\hat{\Theta}) - m \log N,$$

según la convención empleada por `mclust`, donde m representa el número de parámetros.

Un modelo con más componentes suele ajustar mejor los datos, pero también introduce más parámetros. BIC penaliza esa complejidad adicional.

Conexión con aprendizaje máquina

La selección del número de componentes es un ejemplo de selección de modelos. El algoritmo no sólo estima los parámetros para un K fijo, sino que puede ayudar a comparar distintas estructuras probabilísticas.

16. Responsabilidades y medias estimadas

Después del ajuste:

```

1 Z <- fit$z
2
3 MU <- t(
4   fit$parameters$mean
5 )

```

La matriz Z contiene responsabilidades:

$$Z \in \mathbb{R}^{N \times \hat{K}}.$$

Por su parte,

$$MU \in \mathbb{R}^{\hat{K} \times 3}$$

contiene las medias RGB de los componentes.

Cada fila de MU puede verse como un color dominante aprendido.

17. Cuantización MAP

La primera reconstrucción asigna cada píxel al componente de mayor probabilidad posterior:

$$\hat{z}_i = \arg \max_k \gamma_{ik}.$$

Luego reemplazamos x_i por

$$\mu_{\hat{z}_i}.$$

En R:

```

1 k_map <- max.col(Z)
2
3 X_map <- MU[k_map, ]
4
5 img_map <- array(
6   X_map,
7   dim = c(H, W, 3)
8 )

```

17.1. Interpretación como compresión de colores

Si existen $\hat{K}$ componentes, la imagen MAP utiliza únicamente $\hat{K}$ colores representativos, uno por componente.

Así, una imagen que originalmente podía contener miles de valores RGB distintos se reemplaza por una paleta mucho más pequeña.

Interpretación

Esta operación es una forma de **cuantización de color**. Los detalles cromáticos dentro de cada componente se eliminan y se conserva únicamente su color representativo.

18. Reconstrucción mediante media posterior

Existe una alternativa a la clasificación dura. Para cada píxel podemos calcular

$$\hat{x}_i^{\text{soft}} = \sum_{k=1}^{\hat{K}} \gamma_{ik} \mu_k. \quad (8)$$

En forma matricial:

$$X_{\text{soft}} = Z MU.$$

En R:

```
1 X_soft <- Z %*% MU
2
3 img_soft <- array(
4   X_soft,
5   dim = c(H, W, 3)
6 )
```

18.1. MAP frente a promedio posterior

La diferencia conceptual es importante.

Para MAP,

$$\hat{x}_i^{\text{MAP}} = \mu_{\hat{z}_i}.$$

Para la media posterior,

$$\hat{x}_i^{\text{soft}} = \sum_k \gamma_{ik} \mu_k.$$

La primera elige un único centro. La segunda combina varios centros de acuerdo con sus probabilidades posteriores.

Interpretación

Cuando un píxel está claramente asociado con un componente, ambas reconstrucciones serán muy similares. Si el píxel se encuentra en una región ambigua, la reconstrucción blanda conserva esa incertidumbre mediante una combinación de colores.

19. Visualización comparativa

Podemos visualizar simultáneamente la imagen original, la reconstrucción MAP y la reconstrucción posterior:

```
1 par(
2   mfrow = c(1,3),
3   mar = c(1,1,2,1)
4 )
5
6 plot.new()
7 grid::grid.raster(
8   img,
9   interpolate = FALSE
10 )
11 title("Original")
```

```

12
13 plot.new()
14 grid::grid.raster(
15   img_map,
16   interpolate = FALSE
17 )
18 title("MAP")
19
20 plot.new()
21 grid::grid.raster(
22   img_soft,
23   interpolate = FALSE
24 )
25 title("Posterior mean")
26
27 par(mfrow = c(1,1))

```

Al comparar estas imágenes conviene preguntar:

- ¿cuánta variabilidad cromática se perdió?
- ¿las fronteras entre regiones se conservan?
- ¿MAP produce cambios más abruptos?
- ¿la reconstrucción blanda genera transiciones más suaves?

20. Geometría RGB y reducción de dimensión con PCA

Los píxeles viven en un espacio tridimensional. Para inspeccionar su geometría en una gráfica bidimensional podemos utilizar Análisis de Componentes Principales (PCA).

```

1 pc <- prcomp(
2   Ximg,
3   center = TRUE,
4   scale. = TRUE
5 )
6
7 dfp <- data.frame(
8   PC1 = pc$x[,1],
9   PC2 = pc$x[,2],
10  k = factor(k_map)
11 )
12
13 set.seed(1)
14
15 idx <- sample(
16   1:nrow(dfp),
17   min(4000, nrow(dfp))
18 )
19
20 ggplot2::ggplot(
21   dfp[idx,],
22   ggplot2::aes(
23     PC1,
24     PC2,
25     color = k
26   )

```

```

27 ) +
28   ggplot2::geom_point(
29     alpha = 0.6,
30     size = 0.7
31   ) +
32   ggplot2::labs(
33     title =
34       "Colores RGB proyectados a 2D mediante PCA"
35   ) +
36   ggplot2::theme_minimal()

```

20.1. ¿Qué representa esta gráfica?

PCA no realiza el agrupamiento. El agrupamiento ya fue obtenido mediante el GMM. PCA únicamente ofrece una proyección de los datos para facilitar su visualización.

Por tanto, los colores del gráfico indican la clasificación MAP, mientras que las coordenadas PC1 y PC2 representan combinaciones lineales de R , G y B .

Punto de atención

Una proyección bidimensional puede ocultar parte de la separación existente en tres dimensiones. La gráfica PCA debe utilizarse como herramienta descriptiva, no como prueba definitiva de que los grupos están bien separados.

21. Uso de una imagen real

El mismo procedimiento puede aplicarse a una imagen PNG o JPEG.

```

1  # install.packages(c("png", "jpeg"))
2
3  library(png)
4  library(jpeg)
5
6  img_path <- "mi_imagen.png"
7
8  if(grepl(
9    "\\..png$",
10    img_path,
11    ignore.case = TRUE
12  )){
13    img_real <-
14      png::readPNG(img_path)
15  } else {
16    img_real <-
17      jpeg::readJPEG(img_path)
18  }
19
20  H2 <- dim(img_real)[1]
21  W2 <- dim(img_real)[2]
22
23  X2 <- matrix(
24    img_real,
25    ncol = 3
26  )
27
28  fit2 <- Mclust(

```

```

29   X2,
30   G = 3:6
31 )
32
33 Z2 <- fit2$z
34 MU2 <- t(fit2$parameters$mean)
35
36 k2_map <- max.col(Z2)
37
38 img2_map <- array(
39   MU2[k2_map, ],
40   dim = c(H2, W2, 3)
41 )
42
43 img2_soft <- array(
44   Z2 %*% MU2,
45   dim = c(H2, W2, 3)
46 )
47
48 par(
49   mfrow = c(1,3),
50   mar = c(1,1,2,1)
51 )
52
53 plot.new()
54 grid::grid.raster(img_real)
55 title("Original")
56
57 plot.new()
58 grid::grid.raster(img2_map)
59 title("MAP")
60
61 plot.new()
62 grid::grid.raster(img2_soft)
63 title("Posterior mean")
64
65 par(mfrow = c(1,1))

```

21.1. Qué debe analizarse en una imagen real

Después del ajuste, conviene inspeccionar:

1. el número de componentes seleccionado;
2. las medias RGB de los componentes;
3. los pesos estimados;
4. la reconstrucción MAP;
5. la reconstrucción blanda;
6. los píxeles con mayor incertidumbre.

Las medias pueden interpretarse como una paleta estadística dominante, pero no necesariamente corresponden a objetos físicos.

Punto de atención

Un GMM basado únicamente en RGB no utiliza información espacial. Dos píxeles con colores similares pueden quedar en el mismo componente aunque estén muy alejados entre sí en la imagen. Por ello, un grupo cromático no debe interpretarse automáticamente como un objeto o una región conectada.

22. Una limitación fundamental: se ignora la posición

Hasta ahora cada píxel se representa sólo mediante

$$(R, G, B).$$

Las coordenadas espaciales de la imagen,

$$(u, v),$$

no forman parte del modelo.

Esto significa que el GMM identifica *similitud cromática*, no necesariamente *segmentación espacial*.

Una extensión natural consiste en representar cada píxel mediante

$$x_i = (R_i, G_i, B_i, \lambda u_i, \lambda v_i),$$

donde $\lambda > 0$ controla cuánto pesa la proximidad espacial frente a la semejanza de color.

Esta extensión transforma el problema de agrupamiento cromático en uno que también considera estructura espacial.

Conexión con aprendizaje máquina

Esta observación distingue dos objetivos frecuentes:

- **cuantización de color:** interesa reducir la paleta;
- **segmentación:** interesa identificar regiones coherentes de la imagen.

Un GMM en RGB es especialmente natural para el primero. Para el segundo puede ser necesario incorporar posición o modelos espaciales adicionales.

23. Comparación conceptual con k -means

Tanto k -means como GMM pueden utilizarse para agrupar colores, pero no representan el mismo modelo.

En k -means, cada observación se asigna a un único centro:

$$\hat{z}_i = \arg \min_k \|x_i - \mu_k\|^2.$$

En un GMM se calculan probabilidades posteriores:

$$\gamma_{ik} = P(Z_i = k \mid x_i).$$

Además, un GMM puede representar componentes con diferentes dispersiones y correlaciones mediante Σ_k .

Aspecto	<i>k</i> -means	GMM
Pertenencia	dura	probabilística
Centro	centroide	media Gaussiana
Dispersión	implícitamente similar	modelada mediante Σ_k
Forma de grupos	aproximadamente esférica	elipsoidal, según covarianza
Incertidumbre	no explícita	responsabilidades γ_{ik}

24. Evaluación de la reconstrucción

Además de la inspección visual, puede cuantificarse la diferencia entre la imagen original y una reconstrucción.

El error cuadrático medio es

$$\text{MSE} = \frac{1}{3HW} \sum_{i=1}^{HW} \|x_i - \hat{x}_i\|^2.$$

Si los valores están en $[0, 1]$, una forma usual de PSNR es

$$\text{PSNR} = 10 \log_{10} \left(\frac{1}{\text{MSE}} \right).$$

En R:

```

1 mse_rgb <- function(X, Xhat){
2   mean((X - Xhat)^2)
3 }
4
5 psnr_rgb <- function(X, Xhat){
6   mse <- mse_rgb(X, Xhat)
7   10 * log10(1 / mse)
8 }
9
10 mse_map <- mse_rgb(Ximg, X_map)
11 mse_soft <- mse_rgb(Ximg, X_soft)
12
13 psnr_map <- psnr_rgb(Ximg, X_map)
14 psnr_soft <- psnr_rgb(Ximg, X_soft)
15
16 c(
17   MSE_MAP = mse_map,
18   MSE_soft = mse_soft,
19   PSNR_MAP = psnr_map,
20   PSNR_soft = psnr_soft
21 )

```

Interpretación

Estas métricas miden fidelidad numérica respecto de la imagen original. No determinan por sí solas si la agrupación es semánticamente útil. Una reconstrucción con buen PSNR puede no coincidir con las regiones que un observador humano considera relevantes.

25. Preguntas de autoevaluación

Antes de continuar con ejemplos más complejos, el estudiante debería poder responder:

1. ¿Por qué un píxel RGB puede verse como un vector en $\mathbb{R}^3$?
2. ¿Qué representa la media μ_k de un componente?
3. ¿Qué representa Σ_k en términos de variabilidad cromática?
4. ¿Cuál es la diferencia entre π_k y γ_{ik} ?
5. ¿Por qué las responsabilidades de cada píxel suman uno?
6. ¿Qué significa que una responsabilidad sea cercana a 0.5?
7. ¿Por qué el paso M utiliza promedios ponderados?
8. ¿Qué propiedad de la log-verosimilitud se espera durante EM?
9. ¿Cuál es la diferencia entre la imagen MAP y la imagen de media posterior?
10. ¿Por qué un GMM en RGB no constituye necesariamente una segmentación espacial?
11. ¿Qué información ofrece PCA y qué información no ofrece?
12. ¿Cómo se interpreta el número de componentes seleccionado por BIC?

26. Ejercicios propuestos

Ejercicio 1. Segunda iteración de EM.

Utilice $(\pi^{\text{new}}, \mu^{\text{new}}, \Sigma^{\text{new}})$ obtenidos en la primera iteración y vuelva a ejecutar los pasos E y M. Compare las responsabilidades y verifique nuevamente la log-verosimilitud.

Ejercicio 2. Sensibilidad a la inicialización.

Modifique las medias iniciales para que dos componentes comiencen relativamente cerca. Analice si los píxeles cambian de componente durante las primeras iteraciones.

Ejercicio 3. Píxel frontera.

Agregue el color

$$(R, G, B) = (200, 180, 30)$$

y calcule sus responsabilidades. Explique por qué puede presentar mayor incertidumbre que los colores puros.

Ejercicio 4. Número de componentes.

Ajuste por separado modelos con

$$K = 3, \quad 4, \quad 5, \quad 6.$$

Compare BIC, las paletas aprendidas y la calidad visual de las reconstrucciones.

Ejercicio 5. Modelos de covarianza.

Explore en `mclust` distintos `modelNames`, incluyendo estructuras más restrictivas y más flexibles. Compare BIC y describa cómo cambia la geometría de los componentes.

Ejercicio 6. PSNR.

Calcule PSNR para las imágenes MAP y de media posterior. Repita el experimento para distintos valores de K .

Ejercicio 7. Imagen real.

Seleccione una imagen con varios colores dominantes. Ajuste el GMM y describa qué representan las medias μ_k .

Ejercicio 8. Incertidumbre.

Para cada píxel calcule

$$\max_k \gamma_{ik}.$$

Identifique los píxeles con valores más bajos y localícelos visualmente en la imagen.

Ejercicio 9. Entropía posterior.

Para cada píxel defina

$$H_i = - \sum_{k=1}^K \gamma_{ik} \log \gamma_{ik}.$$

Construya una imagen de entropía. Explique qué regiones de la imagen presentan mayor incertidumbre.

Ejercicio 10. Incorporación de posición.

Extienda los vectores RGB a

$$(R, G, B, \lambda u, \lambda v).$$

Pruebe distintos valores de λ y describa cómo cambia el agrupamiento.

27. Resumen conceptual

El procedimiento completo puede sintetizarse de la siguiente forma:

1. representar cada píxel mediante un vector RGB;
2. considerar esos vectores como observaciones en $\mathbb{R}^3$;
3. modelar su distribución mediante un GMM;
4. estimar responsabilidades en el paso E;
5. actualizar pesos, medias y covarianzas en el paso M;
6. iterar hasta alcanzar convergencia;
7. utilizar las medias estimadas como colores representativos;
8. reconstruir la imagen con asignación MAP o media posterior;
9. evaluar el resultado visual y cuantitativamente.

Interpretación

El punto central es que un GMM no “ve” una imagen como lo hace una persona. Observa una muestra de vectores y construye una representación probabilística de su distribución. Su utilidad en imágenes surge al reinterpretar los parámetros estadísticos como colores dominantes y las responsabilidades como grados de pertenencia de cada píxel.

28. Bibliografía recomendada

Referencias

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

- [2] T. Hastie, R. Tibshirani y J. Friedman, *The Elements of Statistical Learning*, 2a ed., Springer, 2009.
- [3] G. McLachlan y D. Peel, *Finite Mixture Models*, Wiley, 2000.
- [4] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.
- [5] L. Scrucca, M. Fop, T. B. Murphy y A. E. Raftery, “mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models”, *The R Journal*, 8(1), 2016.
- [6] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4a ed., Pearson, 2018.